Create a detailed specification and user flow for a mobile app designed for Wi-Fi printing using IP addresses with support for either LPD or Raw printing protocols (the user can select only one protocol at a time). The app should have a bottom navigation bar with three tabs:

- Home Tab (icon: home)
- Allows users to select multiple files, including images and PDFs.
- After file selection, provide basic editing tools for each file on separate screens: Crop, Brightness adjustment, Contrast adjustment, Resize, and options to move the image to top/left or top/right.
- Include a live preview that updates in real-time as edits are made.
- After editing, show print options on a separate screen with live preview updates. Print options must include:
- Layout options: 2 on 1, 4 on 1, 6 on 1, 8 on 1
- Orientation: Portrait or Landscape
- Paper sizes: A4, A3, Legal, A5
- Paper source: Tray (default) or Stack Bypass
- Number of copies (user input)
- Print range selection
- Provide a print preview screen before printing.
- Printer Tab (icon: printer)
- Allow users to enter and save the printer's IP address permanently, until deleted by the user.
- Support setting the protocol type (LPD or Raw) for printing.
- If the user deletes the IP address, remove it from storage.
- Print Queue Tab (icon: three dots)
- Display the current print queue showing jobs with statuses: Printing, Finished, or Error.
- Allow users to cancel any ongoing or queued print job.
- Show the print timing for each job.
- Automatically delete finished jobs from the queue after 2 days.
- Provide an option to manually add jobs to the print queue.

Ensure that all file selection, editing, print options, and print preview screens are separate and clearly organized to handle multiple files independently. The app should provide clear and intuitive navigation between these screens and the three main tabs. Include real-time previews where applicable to enhance user experience.

go with Flutter — it's the best choice here because:
Single codebase for Android & iOS
Strong plugin ecosystem for printing & USB
Better performance than React Native for this use case

Write complete, structured code for the entire app based on the architecture we discussed
Organize it into proper files/folders
Cover all the core modules — discovery, IPP client, document rendering, print job manager, UI screens

Q: Which platform do you want to build for?
A: Both Android & iOS

Q: What features do you want in the first version?
A: Wi-Fi Printing, Usb printing (like noko print)

Q: Do you have an existing GitHub repo to push code into, or starting fresh?
A: I have a repo — will share link

Q: Which framework do you prefer for Both Android & iOS?
A: use which is Best i trust you

Q: For USB printing, which style do you want?
A: Both

Q: What Ui do you want?
A: Clean And beautiful (eg like canon print business,epson wifi print)

wifi_usb_print_app/

```
├── lib/
│   ├── main.dart
│   ├── ui/
│   │   ├── home_screen.dart
│   │   ├── printer_list_screen.dart
│   │   ├── preview_screen.dart
│   │   ├── settings_screen.dart
│   └── domain/
│   │   ├── models/
│   │   │   ├── printer.dart
│   │   │   └── print_job.dart
│   │   ├── usecases/
│   │   │   ├── discover_printers.dart
│   │   │   └── send_print_job.dart
│   └── data/
│       ├── ipp_client.dart        ← Wi-Fi printing core
│       ├── usb_print_client.dart  ← USB/OTG printing core
│       ├── mdns_discovery.dart    ← Printer discovery
│       └── local_db.dart          ← Saved printers
├── android/                   ← USB OTG native code
├── ios/                       ← iOS USB native code
└── pubspec.yaml
```

Discovery Module

User opens app
↓
mDNS scans local network (port 5353)
↓
Finds printers broadcasting _ipp._tcp or _ipps._tcp
↓
Returns list: {name, IP, port, capabilities}
↓
Save to local DB for quick reconnect

Document Processing Module

User selects file (PDF, image, Office doc)
↓
App renders it using PdfRenderer / PDFKit
↓
Applies settings (copies, color, paper size, duplex)
↓
Converts to printer-ready format (raster or PDF)
↓
Chunks into data stream for sending

IPP Communication Module

App builds IPP request packet
↓
Opens TCP socket → Printer IP : Port 631
↓

Sends HTTP POST with IPP payload
↓
Printer responds with job ID + status
↓
App polls job status until complete/error

Print Job Manager

Job created → QUEUED
↓
Connected to printer → SENDING
↓
Printer received → PROCESSING
↓
Done → COMPLETED or FAILED
↓
Log saved to local DB for history

Data Flow (End to End)

[User picks PDF]
↓
[Preview Screen] → user sets copies, color, paper size
↓
[PrintJob object created]
↓
[PdfRenderer] converts to raster images per page
↓
[IppClient] wraps pages into IPP message
↓
[SocketManager] opens TCP to printer IP:631
↓
[Printer] receives, prints, returns status
↓
[UI] shows success / error toast
↓
[Local DB] logs the job